

Object Oriented Programming

Homework 2 - Emulating a 2D Array

(Deadline: 2025/3/24)

1. Question Description:

In C++, multi-dimensional arrays are a fundamental data structure that allows programmers to store and access data in a tabular format. However, internally, multi-dimensional arrays are stored as contiguous blocks of memory, and their indices are converted into single offsets when accessing elements.

In this assignment, we will simulate a two-dimensional (2D) array using a dynamically allocated one-dimensional (1D) array. You will write a set of functions to manage this structure, allowing users to store and retrieve values as if they were working with a normal 2D array.

This approach is useful when working with lower-level memory management, optimizing cache efficiency, or implementing matrix-related operations without direct support for multi-dimensional arrays.

Your task is to implement three functions:

1. `int* create2DArray(int rows, int columns);`
 - This function dynamically allocates a one-dimensional array that can hold $\text{rows} \times \text{columns}$ elements to emulate a two-dimensional array.
 - Input: Two integers `rows` and `columns`, representing the number of rows and columns.
 - Output: A pointer to a dynamically allocated one-dimensional array.
2. `void set(int *arr, int rows, int columns, int desired_row, int desired_column, int val);`
 - This function stores the value `val` at the specified location (`desired_row`, `desired_column`).
 - Input: An array pointer, the total number of rows, the total number of columns, a target row and column, and the value to be stored.
 - Output: The value is stored in the appropriate position in the one-dimensional array. If the indices are out of bounds, print an error message: `Error: Index out of bounds`
3. `int get(int *arr, int rows, int columns, int desired_row, int desired_column);`
 - This function retrieves and returns the value at the specified location (`desired_row`, `desired_column`).

- Input: An array pointer, the total number of rows, the total number of columns, and a target row and column.
- Output: The value at the specified row and column. If the indices are out of bounds, print an error message: `Error: Index out of bounds`. If the value at the position has not been set, print 0.

2. Input and Output Format

input.txt (argv[1]):

- The first line contains two integers:
rows columns
indicating the dimensions of the emulated 2D array.
- The following lines contain commands (set or get), each with the required parameters:
set row column value
get row column
set row column value: Stores value at matrix[row][column]
get row column: Retrieves the value at matrix[row][column]
- All inputs are positive integers.

output.txt (argv[2])

- If the command is set, nothing is printed unless an out-of-bounds error occurs.
- If the command is get, output the retrieved value.
- If an out-of-bounds error occurs (invalid row/column access), print:
`Error: Index out of bounds`

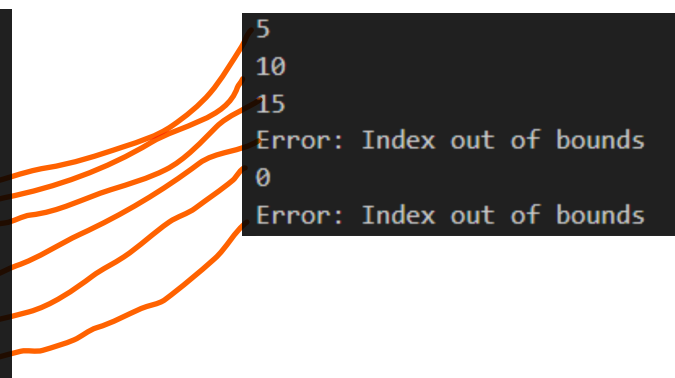
Example Case:

input.txt

```
3 4
set 0 0 5
set 1 2 10
set 2 3 15
get 0 0
get 1 2
get 2 3
set 3 1 20
get 1 1
get 2 4
```

output.txt

```
5
10
15
Error: Index out of bounds
0
Error: Index out of bounds
```



3. Note

- You **must** use dynamic memory allocation (new and delete). Failure to do so will result in a score of 0.
- You are **not allowed** to use built-in 2D arrays (e.g., `int arr[rows][cols]`). Doing so will result in a score of 0.
- Plagiarism will result in a score of 0.
- The submission deadline for the assignment is March 24, at 18:30 (before class). Naming rule: `<StudentID>_HW2.cpp`
- Starting from 8:00 AM tomorrow, you can verify the open cases using the OJ system commands.
`/home/share/2024OOPDEMO HW 02`
- Hidden cases will be demonstrated during the next lab session.